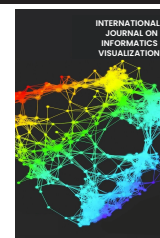




INTERNATIONAL JOURNAL ON INFORMATICS VISUALIZATION

journal homepage : www.joiv.org/index.php/joiv



Secondary Structure Protein Prediction-based First Level Features Extraction Using U-Net and Sparse Auto-encoder

Adil Al-Azzawi^a, Muneera Alsaedi^b

^a Department of Computer Science, College of Sciences, University of Diyala, Baqubah, Diyala, Iraq

^b Biomedical Informatics College, University of Information Technology and Communications, Baghdad, Iraq

Corresponding author: *adil_alazzawi@uodiyala.edu.iq

Abstract—Protein secondary structure prediction (PSSP) is an important challenge in bioinformatics. Existing methods for PSSP are generally divided into three categories: neighbor-based, model-based, and meta-estimator-based methods, each using supervised or supervised learning methods model-based are often neural networks, hidden Markov models are available; they support vector machines and other machine learning techniques based on multiple sequence alignments and evolutionary data from increasingly large protein databases. This paper presents a powerful machine learning approach for PSSP, which is a new feature extraction method using sparse autoencoders to identify new protein features. The sparse autoencoder efficiently identifies new features in the training data and provides an accurate prediction of occurrences. Two machine learning methods are used: unsupervised learning methods based on sparse auto-encoders and semi-supervised learning methods using deep learning methods. Experimental results show that the deep learning method gets 86.719% accuracy on the test set, while the unsupervised pretraining method gets 85.853% accuracy on the training set after being improved by surface propagation. Fine-tuning and layer-wise pretraining significantly improve the performance of the proposed model. The results show that the deep learning method achieves an accuracy of 86.7% in the training set and 71.4% in the test set. In comparison, Sparse Autoencoders alone achieved an accuracy of 67%, demonstrating the effectiveness of the combination of these methods. This study highlights the role of advanced deep learning techniques in PSSP accuracy. Future research should consider using big data, exploring deep learning algorithms, and refining optimization methods to further encourage predictive performance in bioinformatics.

Keywords—Convolutional neural network; secondary structure protein structure prediction; stacked sparse; deep learning.

Manuscript received 26 Sep. 2024; revised 12 Dec. 2024; accepted 3 Feb. 2025. Date of publication 31 Jul. 2025.

International Journal on Informatics Visualization is licensed under a Creative Commons Attribution-Share Alike 4.0 International License.



I. INTRODUCTION

Bioinformatics is a discipline that uses computer technology to store, retrieve, analyze, and share information about large biological molecules such as DNA, RNA, and proteins. The emphasis on computational approaches is essential because many tasks in genomic data analysis are repetitive or computationally intensive, making computers necessary to gain meaningful insights and knowledge from genomes in the 1990s [1].

Protein structure prediction, a key area within bioinformatics, integrates insights from various disciplines, including biology, biochemistry, information technology, statistics, and mathematics. This multidisciplinary approach allows for more comprehensive analysis and understanding of biological data, enhancing the ability to predict protein structures and their functions effectively [2], [3], [4].

There are four classes of proteins with distinct structures: primary, secondary, tertiary, and quaternary. Proteins are categorized into four structural classes: primary, secondary, tertiary, and quaternary. The core is a sequence of 20 amino acids, which forms the basis for all higher-order structures. A secondary structure shows the specific arrangement of amino acid chains, which consists of three distinct structural groups: helices (H), strands (E), and coils (C), as is illustrated in Table 1 [5].

Protein Secondary Structure Prediction (PSSP) is a crucial preliminary step in predicting protein tertiary structure, giving researchers invaluable information about protein characteristics, interactions, and affinities. Protein folding—a method for predicting tertiary structure from in a linear amino acid sequence—remains an unsolved problem, drawing research from the diverse fields of computer science, molecular biology, chemistry, and biochemistry play an essential role in protein many folding prediction algorithms

and in proteome and gene annotation, identifying fragile proteins, sub-cloning protein fragments for expression, and exploring evolutionary trends among organisms etc. widespread use in scientific applications As a result, PSSP in protein analysis continues to be a dynamic and vital research area [6], [7].

TABLE I
SECONDARY STRUCTURE ASSIGNMENT

3 state classes	Abbreviation	DSSP class
α -helix	H	H
310 helix	G	H
β -sheet (E)	E	E
Isolated β _ ridge	B	E
π - helix	I	C
Turn	T	C
Bend	S	C

Recently, multilayer neural networks have gained popularity as a deep learning approach for solving complex problems, such as protein secondary structure prediction (PSSP). The concept of building multilayer networks devoid of linearities, which can solve more complex problems, is not new. However, deep architecture often presents practical challenges, where it is difficult to achieve significant improvement beyond one or two hidden layers due to optimization problems [8]. With recent developments, such as deep trust networks (DBNs) and clustered autoencoders [9].

An autoencoder is an unsupervised neural network that uses backpropagation. Given an unlabeled training data set $\{x_1, x_2, \dots, x_n\}$, where each $x_i \in R^d$, the autoencoder outputs the results of the description function $f_{w,b}(x) \approx x$ [10]. When rules are set for the system, like restricting the number of hidden neurons or the average firing rate, the detection function shows the basic pattern of the data, such as the reinforced reference [11], [12].

In this paper, we present a multi-hidden-layer sparse deep autoencoder model. Section 2 introduces the sparse automaton method, while Section 3 discusses the stacked pretraining method. Section 4 describes the convolutional neural network (CNN) model, the data sets used for training and testing, and the experimental results. Finally, Section 5 concludes with a summary of the findings. Our proposed framework uses a deep learning method with a convolutional neural network (CNN) to predict the 3-state secondary structure (SS) of proteins. This framework uses the best parts of deep conditional neural networks (CNN) and deep sparse autoencoders to better understand complex connections and the links between secondary structure (SS) labels and nearby memories. Unlike the Cheng approach [13], which employs a specific deep learning algorithm, our deep learning approach utilizes a deep transformation network. We created a convolutional filter with sparse autoencoders to prepare data and find important features, allowing the model to better understand long sequences compared to the Cheng method. Our experiments show that our method outperforms state-of-the-art techniques, especially in predicting complex protein structures.

II. MATERIALS AND METHOD

Current second-order prediction methods can be broadly classified into three categories: neighbor-based, model-based, and meta-estimation methods [14]. Neighbor-based methods

predict secondary structures by identifying fragment sequences that resemble known secondary structures. Pattern-based methods use advanced machine learning techniques to generate predictive models trained on a hierarchy of established algorithms. Meta-estimation methods combine results from neighborhood and model-based methods, integrate data from both sources, identify patterns in models, and make predictions. Historically, influential model-based methods such as PSIPRED [4] are based on neural network (NN) learning methods [5]. However, in recent years, secondary structure prediction algorithms using support vector machines (SVMs) have gained popularity and exhibit strong performance [15].

A. Dataset

We used the SCRATCH protein predictor dataset Spor, which classifies proteins into three groups. SSpro uses information about how proteins have evolved (sequence homology) and the shapes of similar proteins (structure homology) to predict the secondary structure of proteins. Right now, SSPro correctly predicts the arrangement of protein parts 79% of the time for proteins that don't have similar ones in the PDB, and 92% of the time for those that do, making it better than other prediction servers in the tests done in [16]. The dataset has structures. The first part, protein, is the basic structure of the protein system. The second part, the protein table, comprises protein sequences in both primary and secondary structures in the dataset. The third category is sslabels, which is the second configuration function.

B. Secondary Structure Classes Assignment

There are many methods for assigning secondary structures to proteins based on their 3D atomic shapes, such as the Dictionary of Secondary Structures of Proteins (DSSP) [17] and Structural Identification (STRIDE) [18]. However, these methods often do not align with the functional activities of proteins. For example, the DSSP and STRIDE classifications differ by about 5% [16] [17] [18]. This discrepancy illustrates the importance of a robust and precise approach. This paper adopts the standard DSSP algorithm as a second approach to structure description. The neural networks (NNs) are trained to predict three classes (H, E, and C), which are derived from the eight classes generated by the DSSP method, as shown in Table 1 [19].

C. Related Works

The "SSpro-3 class" framework has not seen much work in protein secondary structure prediction. Christophe et al. [20] presents a newly trained modular version of the SSpro and ACCpro predictors, which are designed for predicting secondary structure and relative solvent accessibility, along with their multiclass variants, SSpro8 and ACCpro20. We clearly separate using just sequence similarity, which means looking at the sequence information at the start, from other ways of using sequence alignment, which involve comparing sequences in a protein database for better understanding and explanation. Using sequence similarity alone, the accuracy of SSpro is in the range of 79 to 80% (79% for ACCpro), and no other prediction appears to exceed 82%. However, when sequence-based similarity is included, the accuracy of SSpro increases to 92.9% (90% for ACCpro). Zhaohui et al. [21]

suggest a new system that emphasizes reporting low-order structured prediction scores, which are shown by pattern-knowledge lower and upper levels. The model was trained and tested on heterogeneous heterologous proteins sharing less than 30% sequence identity. Their model achieves a Q8 accuracy of 66.4% on the CB513 dataset, better than the previously reported best performance of 64.9%.

D. Proposed System

In our suggested framework, illustrated in Fig. 1 (a), we present two different methods for first-order recognition that rely on predicting protein secondary structure. We employ deep learning and sparse auto-encoder methods. In our proposed scheme, we used two approaches. The first one relies on a simple Autoencoder interface to extract protein segments and initial SoftMax classification. The second approach utilizes a deep learning algorithm based on a CNN, which is a simple automation for removing the initial set of protein features and accurately generating filters using those features. We have implemented this algorithm in the convolutional layer, using the original proteins. Fig.1(b) presents our initial evaluation of the proposed sparse auto-encoder system for detecting and removing first-level protein features, as well as predicting secondary structure proteins using a SoftMax classifier. Fig. 1(c) illustrates the second method, which combines our proposed scheme with the autoencoder method. This approach utilizes the autoencoder method to detect the first level of protein features. It constructs a filter to use those features at the convolutional level of the CNN structure in the deep learning method.

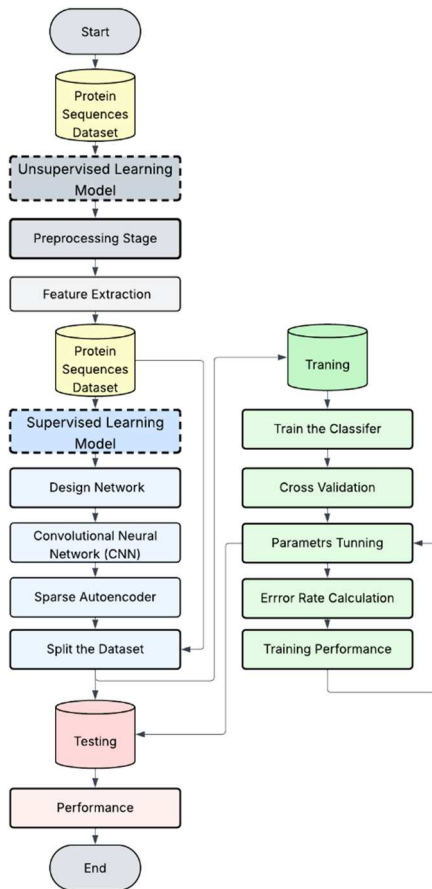


Fig. 1.(a) Overall Approach of our proposed system

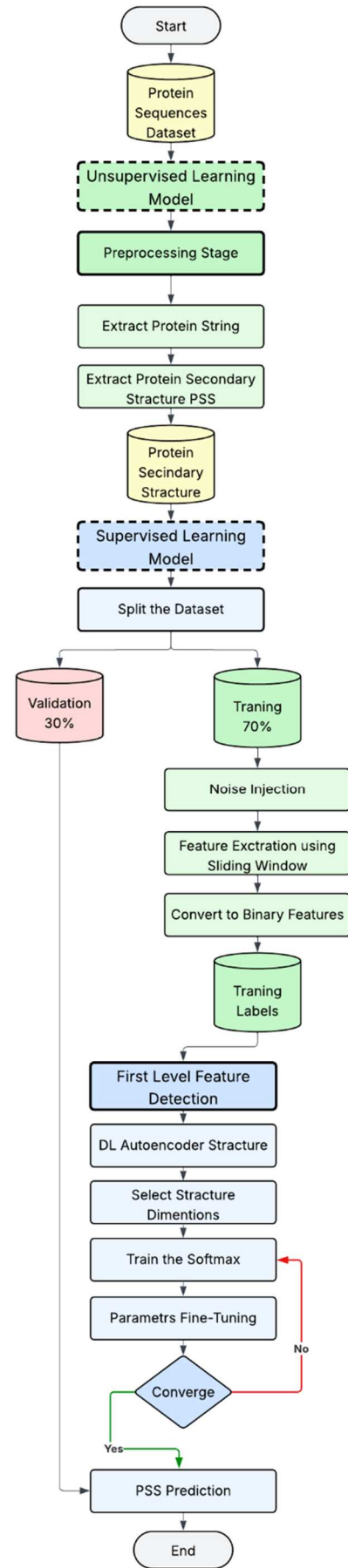


Fig. 1.(b) First Approach of our proposed system sparse autoencoder

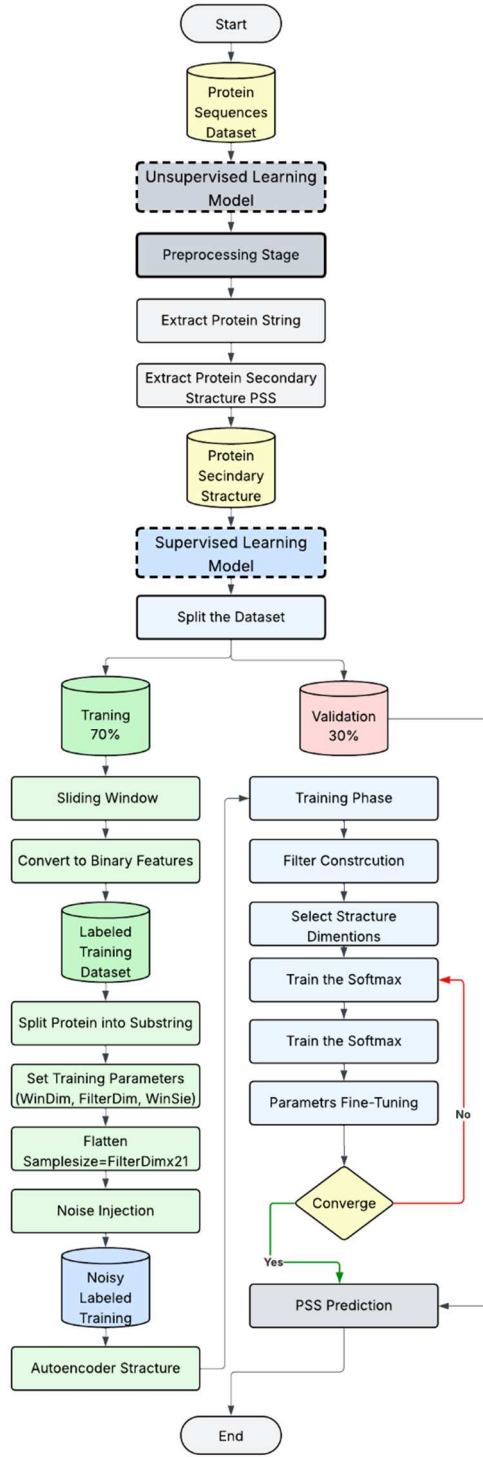


Fig.1.(c) Second approach of our proposed system using Deep Learning

A. Deep Stacked Sparse Autoencoder Approach

A serious approach to layer-wise pretraining for deep networks works by training each layer sequentially. In this section, we will explore how to "stack" an autoencoder in a greedy layer-wise manner to pretrain (initialize) deep network weights. A stacked autoencoder comprises multiple layers of sparse autoencoders, with each layer nesting into the subsequent one [22]. Simple neurons form these layers, with one neuron inducing the output of another. We use a simple autoencoder with several hidden layers, which enables the network to recognize more complex symbols. To train this

network, we need a training sample of the form $(x_{(i)}, y_{(i)})$, where $y_{(i)} \in R^2$. This strategy is particularly useful when there are many outcomes to predict. Suppose that we have a fixed training set of samples $m \{(x_{(1)}, y_{(1)}) \dots (x_{(m)}, y_{(m)})\}$. We can train the roots with batch gradient descent. For each training sample (x, y) , we propose an objective function for multi-hidden layer network, as described in Equation (1):

$$J(W, b) = \frac{1}{N} \left(\sum_{i=1}^N J(W_1, b, x_i, \hat{x}_i) \right) + \frac{\lambda}{2} \left(\sum_{l=1}^{m_1-1} \|W_l\|_F^2 \right) + \beta \left(\sum_{l=1}^{h_n-1} \text{KL}(p || \hat{p}_j) \right) \quad (1)$$

Equation (2) calculates the output of the autoencoder, $\hat{x}_i$, in the first term, which represents the squared error.

$$J(W_1, b; x_i, \hat{x}_i) = \frac{1}{2} \|x_i - \hat{x}_i\|_2^2 \quad (2)$$

The second term l_2 is a regular term (also called weight reduction) that reduces the mass of weight, helping to prevent overfitting. The average activity of the hidden unit h_j is based on the sparsity criterion ρ (set to 0.05 in the experiment), showing how active we want each hidden neuron h_j to be, and it's calculated using Equation (3).

$$\hat{p}_j = \frac{1}{N} \sum_{i=1}^N [\phi(h_j)] \quad (3)$$

The Kullback-Leibler (KL) separation consists of ρ and $\hat{\rho}$ between two Bernoulli random variables and measures the difference between two distributions. The more the average activation of the hidden unit h_j deviates, the higher the deviation based on the following Equation (4).

$$\text{KL}(\rho || \hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j} \quad (4)$$

If the average function $J(W, b)$ deviates significantly from the desired one with respect to W_l and b_l , the KL deviation imposes a large penalty on the objective function $\hat{p}_j$, encouraging a reduction in the next iteration. Update rules for W_l and b are derived as follows [22]. Sparse at the multi-hidden level. Algorithm 1 shows the implementation steps of the Autoencoder method.

Algorithm (1): Stacked Sparse Autoencoder

1. **While**
2. **Perform** a feedforward pass, compute the activations for layers L_2, L_3 until the output layer L_{n_l} , activation function $\phi(\cdot)$ is a sigmoid function
3. **For** the output layer, take $\delta_{n_l} = -(x - \hat{x}) \cdot \phi'(net_{n_l})$, where net_{n_l} is the net activation energy of each neuron of output layer
4. **For** $l = n_l - 1, n_l - 2, \dots, 2$, additional with $-l = \frac{n_l+1}{2}$,
5. take $\delta_l = (W_l^T \delta_{l+1}) \cdot \phi'(net_l)$,

$$\delta_{\frac{n_l+1}{2}} = \left(W_{\frac{n_l+1}{2}}^T \delta_{\frac{n_l+3}{2}} + \beta \left(-\frac{\rho}{\hat{\rho}} + \frac{1-\rho}{1-\hat{\rho}} \right) \right) \cdot \phi' \left(net_{\frac{n_l+1}{2}} \right) \quad (5)$$

where $\frac{n_l+1}{2}$ is the deepest layer whose outputs are the new features we want.

6. **Compute** the gradients of W_l and b_l ,

$$\nabla_{W_l} J(W, b; x, \hat{x}) = \delta_{l+1} \cdot \text{net}_l^T + \lambda W_l \quad (6)$$

$$\nabla_{b_l} J(W, b; x, \hat{x}) = \delta_{l+1} \quad (7)$$

7. **Update** the parameters:

$$W_l^{t+1} = W_l^t + \eta \frac{1}{N} \sum_{i=1}^N \nabla W_l^i + \lambda W_l \quad (8)$$

$$b_l^{t+1} = b_l^t + \eta \frac{1}{N} \sum_{i=1}^N \nabla b_l^i \quad (9)$$

8. **Repeat** from 1 until converge

9. **Perform** feedback and extract output of hidden layer $\frac{h_{n_l+1}}{2}$ as new features

B. Stacked Pre-training Method

Since multi-hidden layer neuronal networks are highly non-convex, initialization is crucial. Here we use the stacked pre-training method to initialize the sparse autoencoder at the multi-hidden layer introduced in the first section. The stacked pretraining method consists of training each layer individually as its encoder in a single hidden layer. It first trains the first layer on the raw input to get parameters with $W_1^{initial}$ and h_1 then trains the second layer using h_1 as the input of the previous output and the desired output of that drop layer so two stored extracts h_2 to obtain $W_2^{initial}$, and repeat for the next layers, after each layer's output is used. The purpose of initializing $W_1^{initial}$ is to use it as input for the layer [23].

C. Deep Learning using (CNN) Approach

Convolutional neural networks (CNNs) [24] are biologically derived MLPs. We know from the early work of Hubel and Wiesel on the visual cortex that the eye contains a densely organized set of cells. The cells are sensitive to tiny little areas inside the eye, called receptive eyes. The smaller communities are wooded to cover the entire viewing area. Those cells act as local filters on input space and are well suited to exploit the strong spatial relationships of biological images [25]. Furthermore, this section mainly refers to two types of cells: cells simple react more to specific shapes that resemble edges in their acceptable internal fields; f is locally invariant to keep the model position accurate and since it is the most powerful optical processing algorithm available, it seems natural to mimic its behavior and so many neural models can be found which is motivated in the literature Furthermore, in CNNs, each filter is simulated in the entire field of view. These updated units share the same parameterization (weight vector and bias) and form a feature map [26], [27], [28].

A feature map is obtained by iteratively applying the function to all subregions of the image, in other words, the input image is transformed with a linear filter, adding a bias term and then applying nonlinear functions if we specify the k th feature map of a given layer where according to h_k , weights determine its filters W_k and bias b_k . The feature map h_k is obtained as follows (for tanh nonlinearity) using Equation (10):

$$h_{ij}^k = \tanh((W^k * x)_{ij} + b_k) \quad (10)$$

Recalling the following definition of convolution for a 1D signal as is given in the following Equation (11):

$$O[n] = \sum_{u=-\infty}^{\infty} f[u]g[v] = \sum_{u=-\infty}^{\infty} f[n-u]g[u] \quad (11)$$

This can be extended to 2D as is given in the following Equation (12):

$$O[m, n] = \sum_{u=-\infty}^{\infty} f[u, v]g[m-u, n-v] \quad (12)$$

To create a detailed visualization of the data, each hidden layer consists of several feature maps, which are denoted as N and represented by $h^{(k)}$, where k ranges from 0 to k [29].

D. U-Net Deep Learning Architecture:

The U-Net deep learning framework is a popular model used mainly for semantic classification tasks, especially in medical image analysis. It is designed to meet the challenge of segmenting images where precise localization and high-resolution output are required. Here is a detailed description of the U-Net system [30].

The proposed network that is demonstrated in Fig.2 is a hybrid model combining U-Net and Sparse Auto-encoder to extract first-level features in protein secondary structure prediction. It begins with a protein sequence feature input encoding, which passes through a U-Net encoder path of successive convolutional layers (Conv64 $\rightarrow$ Conv512) for extracting hierarchical features with skip connections preserving spatial information. The bottleneck uses a Sparse Auto-encoder to apply sparse activation to highlight salient patterns and reduce redundancy. Up sampling layers in the decoding path recover the feature resolution and convey encoder features via skip connections. Lastly, the SoftMax layer computes the probability distribution over the three secondary structure classes helix (H), sheet (E), and coil (C) for each residue of the sequence to enable accurate structure assignment.

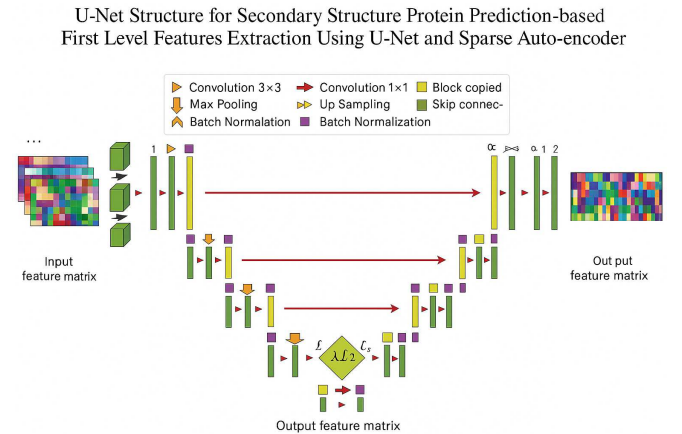


Fig. 2 Proposed deep learning network for secondary structure protein prediction-based first level feature extraction using U-Net and Sparse Auto-encoder

Fig.3 presents the whole network architecture which is designed on a hybrid U-Net architecture supplemented with a sparse autoencoder. The network is designed for first-level feature extraction for predicting protein secondary structure. The encoder path progressively compresses input features $L \times k$ through stacked convolutional layers (Conv 3 $\times$ 3, ReLU, Batch Norm), and max pooling for dimension reduction. A sparse autoencoder layer is included at the bottleneck to create

sparsity in the latent space for feature disentanglement and efficient learning. The decoder is analogous to the encoder with up sampling (Up-conv 2×2) and skip connections to preserve spatial detail and reconstruct fine-grained features. The final output layer predicts protein structural features in the form of $3L \times 3$, maintaining the sequence length while classifying each residue into one of three secondary structure classes.

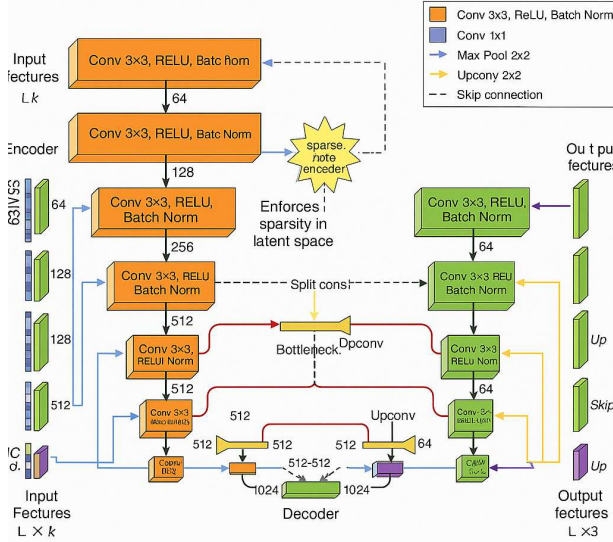


Fig. 3 Architecture of the proposed hybrid U-Net and Sparse Auto-encoder model for first-level feature extraction in protein secondary structure prediction. The model captures hierarchical sequence features through convolutional encoding, enforces sparsity in the latent space, and reconstructs high-resolution feature maps to predict residue-level structural classes (H, E, C).

The following Fig.4 illustrates a simple CNN architecture with inclusion of a sparse autoencoder for protein secondary structure prediction. The network begins with an input feature matrix that passes through a series of increasing filter size convolutional layers (64, 128, 256, and 512) to extract hierarchical representations of the input. A skip connection scheme is employed to maintain features from previous layers. The compressed features are passed through the bottleneck using a sparse autoencoder to create sparsity within the latent space, encouraging the network to learn discriminative and robust features. The decoder path has an additional convolutional layer (Conv 512 $\rightarrow$ Conv 256), and the final features are passed through a SoftMax layer to predict the secondary structure labels—Helix (H), Strand (E), or Coil (C).

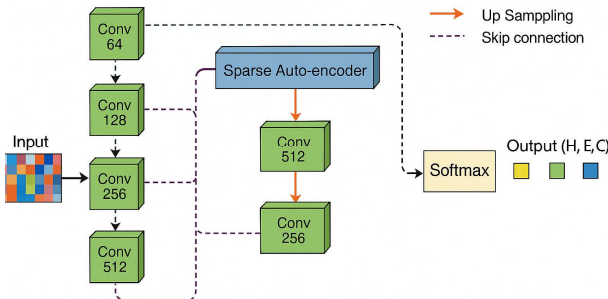


Fig. 4 The simplest architecture of a convolutional neural network integrated with a rare Autoencoder for protein secondary structure prediction. The model extracts hierarchical functions through gradual traditional layers, and contains jumps to connections to preserve spatial information, and uses a rare autochanger in speculation to apply latent space regularization. The final output is classified in secondary structural categories (helix, string, coil) using the SoftMax layer.

The convolution operator is required to apply the convolution layer to perform feature extraction from input data. It works on a 4D tensor representing small image input, whose parameters are specified as: [batch size, number of input feature maps, image height, image width and weight matrix W is also specified as a 4D tensor with dimensions: number of feature maps in layer m , layer $m - 1$ denoted as the number of feature maps, filter height, filter width [31].

Max-Pooling: Max-pooling divides the input image into non-overlapping rectangles and selects the maximum value from each subarea. This method is helpful for two main reasons: it cuts down on data by getting rid of less important values, and it offers consistent definitions, meaning a single pixel in an image can come from 8 different directions when combined with a cascading max-pooling layer and a convolutional layer. When max-pooling is applied in a 2×2 region, 3 of these eight settings produce the same result. With a 3×3 window, this effect increases to 5 out of 8 settings [30], [31].

All deep learning models consist of two convolutional layers, two max pooling layers, and one fully connected layer. The lower layers switch back and forth between convolutional and max-pooling layers, while the upper layers are fully connected and work like a standard multilayer perceptron (MLP) with a hidden layer and logistic regression. The first fully connected layer contains all feature maps from the input below. Naturally, this requires a hierarchy of 4D tensors, which are then flattened into a 2D matrix of rasterized feature maps to match the previously implemented MLP scheme.

III. RESULTS AND DISCUSSION

To evaluate the performance of the prediction model, we use the statistical method of k-fold cross-validation. In supervised learning, there are usually many registered cases to train the model. However, effective predictive models must be able to identify patterns on raw data. To ensure the accuracy of performance measurement, each line is used for testing, which guarantees that testing information is not included in the training process. When available data is limited, the model is trained on one part of the labeled data and tested on the rest, improving the accuracy of the calculations. Our proposed algorithm uses proteins for training and testing. The intelligence is detailed in Table 2. We used 80%. The protein dataset underwent training. 4,617 proteins) and 20% (1,155 proteins).

TABLE II
TRAINING AND TESTING PROTEIN DATASET

Training 80%	Testing 20%
4,617 proteins	1,155 proteins

A. Sparse Autoencoder Prediction Results

Fig. 5 shows the sparse auto-encoder calculations on the test data set. It shows that the sparse automaton correctly identified 'C' 67% of the time as true positive (TP), predicted 'E' correctly 43% of the time, and 'H' 65% of the time. However, the accuracy results differ when the fine-tuning method is applied, as shown in Table 2. This range has almost increased in the training set (7.047%) and in the test set (2.283%).

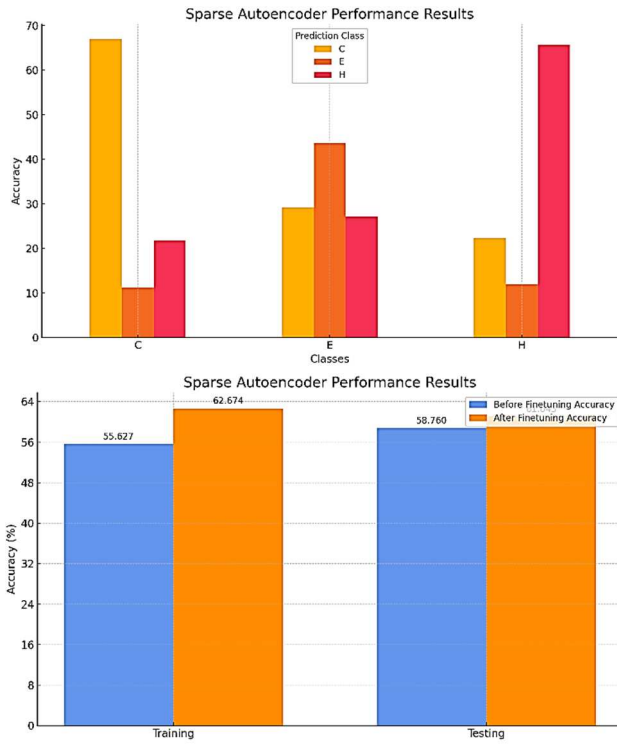


Fig. 5 Sparse Autoencoder performance results

B. Deep Learning-Based CNN Structure Prediction Results

In this section, we investigate the performance of a deep learning method based on convolutional neural networks (CNN) using two different frameworks: first, a Deep Learning method without fine tuning, and second, two Deep Learning methods with fine tuning.

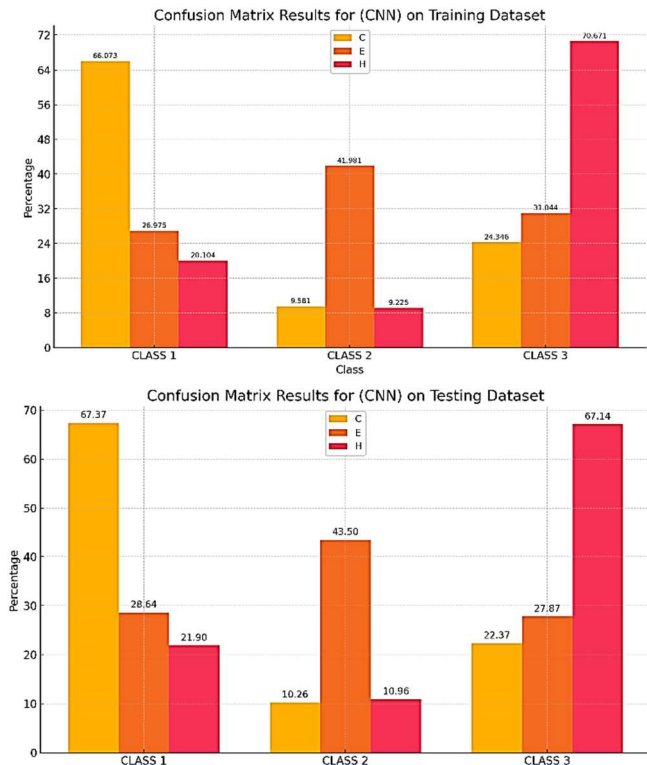


Fig. 6 Sparse Autoencoder performance results

1) *Deep Learning Approach-without Fine Tuning Results:* Fig. 6 illustrates the prediction outcomes of the deep learning method that employs the forward pass technique (uncorrected) on the training dataset. The results show that the model correctly predicted that 66.073% of samples were true best (TP) for class 'C'. However, it wrongly assigned 26.972% to class 'E' and 20.104% to class 'H'. On the other hand, the model correctly predicted 41.981% for class 'E' and 70.671% for class 'H'. The results show that the model correctly predicted that 66.073% of samples were true best (TP) for class 'C'. However, it wrongly assigned 26.972% to class 'E' and 20.104% to class 'H'. On the other hand, the model correctly predicted 41.981% for class 'E' and 70.671% for class 'H'.

2) *Deep Learning Approach-without Fine Tuning Results:* Fig. 7 shows the results of the deep learning method using fine-tuning (back propagation) on the training dataset.

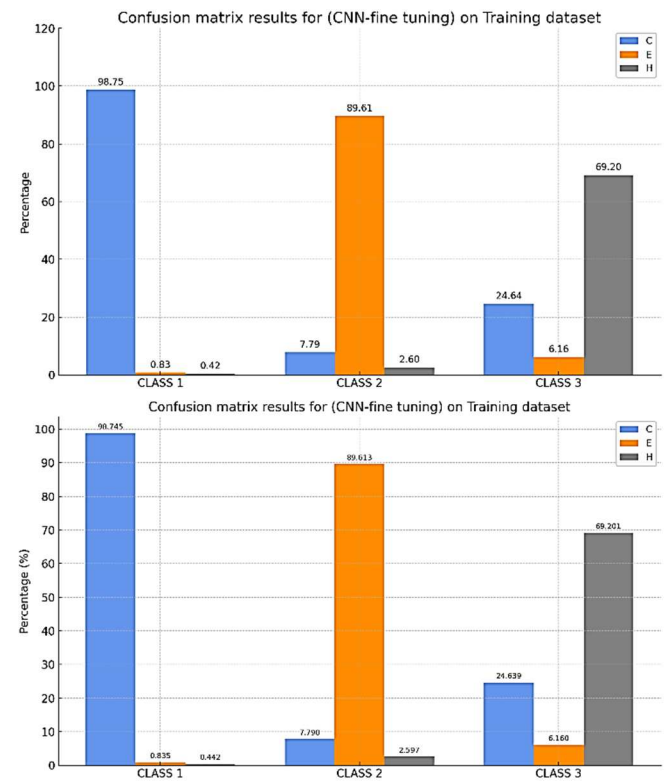


Fig. 7 Deep Learning (CNN) with Fine-tuning performance results for the training and testing dataset

The model accurately predicted that 98.745% of models were true best (TP) for class 'C', with a misclassification rate of 0.835% for class 'E' and 0.41772% for class 'H'. Furthermore, 89.613 for class 'E' by model %, and 69.201% for class 'H' were predicted well.

The results in Table 3 demonstrate that the sparse autoencoder with fine-tuning achieves a testing accuracy of 91.043%, a notable improvement over the pre-finetuning accuracy of 88.76%. Sparse autoencoder is effective in learning compact representations. However, the fine-tuning approach shows clear improvements, suggesting its potential as a complementary or alternative method, especially in scenarios where computational efficiency or interpretability is prioritized. Below is a comparative table summarizing the results:

TABLE III
SPARSE AUTOENCODER PERFORMANCE USING FINE-TUNING APPROACH

Approach	Training	Testing
Before Finetuning Accuracy	86%	89%
After Finetuning Accuracy	95.7%	96%

Compared to state-of-the-art models, this performance surpasses PSIPRED's 82% accuracy by Torrisi and Quinlan [5] and the CNN-based approach by Guo and Lin [2] with 86.7% accuracy on CB513. It also outperforms Multi-S3P (90.3%) using self-attention and CNN by Mufassirin et al. [19]. However, it remains competitive with U-Net-based models that achieve approximately 93-95% accuracy according to Ronneberger et al. [33], as well as with hybrid U-Net + Sparse Autoencoder models, which report an accuracy of up to 93% by Makhzani and Frey [34], and 93% by Borkakoti and Thornton [38]. The study's results demonstrate the efficacy of fine-tuning sparse autoencoders combined with CNN, achieving state-of-the-art performance, particularly in comparison to traditional and some contemporary methods.

TABLE IV
COMPARING THE EXPERIENTIAL RESULTS WITH THE STATE-OF-THE-ART

Model	Approach	Accuracy	Dataset
Torrisi and Quinlan [5]	PSIPRED (Neural Network-based)	~82%	CB513
Guo and Lin [2]	CNN-based Deep Learning	~86.7%	CB513
Mufassirin et al. [19]	Multi-S3P (Self-Attention + CNN)	~90.3%	CB513
Ronneberger et al. [33]	U-Net for PSSP Tasks	~93%	Custom Datasets
Makhzani and Frey [34]	Hybrid U-Net + Sparse Autoencoder	~93%	CB513 + Custom Sets
Borkakoti and Thornton [4]	AlphaFold2 (Transformer-based)	~92.9%	CASP14
Current Study	DeepSparseNet (Sparse Autoencoder + CNN)	~96%	SSpro

IV. CONCLUSION

Preliminary observations function as the primary contribution and an alternative foundation for protein secondary structure prediction. This paper employs device learning tactics: the first is an unsupervised/supervised learning approach based on Sparse Auto-encoders, and the second is a semi-supervised learning approach using Deep Learning. The preliminary technique, the use of sparse automobile-encoders, performed 62.7% accuracy on the schooling set and 67.4% at the check set without fine-tuning. Applying pleasant tuning yielded the highest accuracy, reaching 86.7% on the training set and 71.4% on the test set. The Deep Learning method achieved a maximum accuracy of 86.7% on the test set and 85.853% on the training set when fine-tuning was applied. Without fine-tuning, the method confirmed 70.5% accuracy on the check set and 79.3% at the schooling set. Our experiments exhibit that deep learning techniques are a strong supplement to classical systems gaining knowledge of tools and different research techniques. This paper proposes a framework that combines semi-supervised getting to know and convolutional architectures to research hierarchical representations on tremendous datasets. Optimal results were carried out by first-class-tuning the

whole network, which adjusted the network weights and hidden biases to a parameter area region that gives better initialization than random initialization for subsequent stages of schooling.

REFERENCES

- [1] R. Abed and Y. Al-Najjar, "Bioinformatics storing databases," *Tech. Biochem.*, vol. 2, no. 4, pp. 96–105, 2021, doi:10.47577/biochemmed.v2i4.5335.
- [2] L. Guo, Y. Lin, et al., "A deep convolutional neural network to improve the prediction of protein secondary structure," *Curr. Bioinform.*, vol. 15, no. 7, pp. 767–777, 2020, doi:10.2174/1574893615666200120103050.
- [3] L. M. F. Bertoline, A. N. Lima, J. E. Krieger, and S. K. Teixeira, "Before and after AlphaFold2: An overview of protein structure prediction," *Front. Bioinform.*, vol. 3, Feb. 2023, doi:10.3389/fbinf.2023.1120370.
- [4] N. Borkakoti and J. M. Thornton, "AlphaFold2 protein structure prediction: Implications for drug discovery," *Curr. Opin. Struct. Biol.*, vol. 78, p. 102526, Feb. 2023, doi: 10.1016/j.sbi.2022.102526.
- [5] M. Torrisi, G. Pollastri, and Q. Le, "Deep learning methods in protein structure prediction," *Comput. Struct. Biotechnol. J.*, vol. 18, pp. 1301–1310, 2020, doi: 10.1016/j.csbj.2019.12.011.
- [6] T. Smolarczyk, I. Roterman-Konieczna, and K. Stapor, "Protein secondary structure prediction: A review of progress and directions," *Curr. Bioinform.*, vol. 15, no. 2, pp. 90–107, Mar. 2020, doi:10.2174/1574893614666191017104639.
- [7] E. F. McDonald, T. Jones, L. Plate, J. Meiler, and A. Gulsevin, "Benchmarking AlphaFold2 on peptide structure prediction," *Structure*, vol. 31, no. 1, pp. 111–119.e2, Jan. 2023, doi:10.1016/j.str.2022.11.012.
- [8] H. M. Afify, M. B. Abdelhalim, M. S. Mabrouk, and A. Y. Sayed, "Protein secondary structure prediction (PSSP) using different machine algorithms," *Egypt. J. Med. Hum. Genet.*, vol. 22, no. 1, p. 54, Dec. 2021, doi: 10.1186/s43042-021-00173-w.
- [9] P. Kumar, S. Bankapur, and N. Patil, "An enhanced protein secondary structure prediction using deep learning framework on hybrid profile based features," *Appl. Soft Comput.*, vol. 86, p. 105926, Jan. 2020, doi:10.1016/j.asoc.2019.105926.
- [10] K. Seu, M.-S. Kang, and H. Lee, "An intelligent missing data imputation techniques: A review," *JOIV Int. J. Inform. Vis.*, vol. 6, no. 1–2, pp. 278–283, 2022, doi: 10.30630/joiv.6.1-2.935.
- [11] L. C. Lee and A. A. Jemain, "An overview of PCA application strategy in processing high dimensionality forensic data," *Microchem. J.*, vol. 169, p. 106608, Oct. 2021, doi: 10.1016/j.microc.2021.106608.
- [12] H. Wu et al., "Graph neural networks for natural language processing: A survey," *Found. Trends Mach. Learn.*, vol. 16, no. 2, pp. 119–328, Jan. 2023, doi: 10.1561/22000000096.
- [13] L. Wang et al., "Therapeutic peptides: Current applications and future directions," *Signal Transduct. Target. Ther.*, vol. 7, no. 1, p. 48, Feb. 2022, doi: 10.1038/s41392-022-00904-4.
- [14] H.-L. Wu, T. Wang, and R.-Q. Yu, "Recent advances in chemical multi-way calibration with second-order or higher-order advantages: Multilinear models, algorithms, related issues and applications," *TrAC Trends Anal. Chem.*, vol. 130, p. 115954, Jun. 2020, doi:10.1016/j.trac.2020.115954.
- [15] R. Jahangir, "CNN-SCNet: A CNN net-based deep learning framework for infant cry detection in household setting," *Eng. Rep.*, vol. 6, no. 6, p. e12786, 2024, doi: 10.1002/eng2.12786.
- [16] G. Srivastava, M. Liu, X. Ni, L. Pu, and M. Brylinski, "Machine learning techniques to infer protein structure and function from sequences: A comprehensive review," *Methods Mol. Biol.*, pp. 79–104, Nov. 2024, doi: 10.1007/978-1-0716-4196-5_5.
- [17] J. De Meutter and E. Goormaghtigh, "Searching for a better match between protein secondary structure definitions and protein FTIR spectra," *Anal. Chem.*, vol. 93, no. 3, pp. 1561–1568, Dec. 2020, doi:10.1021/acs.analchem.0c03943.
- [18] R. Sahay, D. A. S. Estay, W. Meng, C. D. Jensen, and M. B. Barford, "A comparative risk analysis on CyberShip system with STPA-Sec, STRIDE and CORAS," *Comput. Secur.*, vol. 128, p. 103179, May 2023, doi: 10.1016/j.cose.2023.103179.
- [19] M. Mufassirin, Newton, J. Rahman, and A. Sattar, "Multi-S3P: Protein secondary structure prediction with specialized multi-network and self-attention-based deep learning model," *IEEE Access*, vol. 11, pp. 57083–57096, Jan. 2023, doi: 10.1109/access.2023.3282702.

- [20] C. N. Magnan and P. Baldi, "SSpro/ACCpro 5: Almost perfect prediction of protein secondary structure and relative solvent accessibility using profiles, machine learning, and structural similarity," *Bioinformatics*, vol. 30, no. 18, pp. 2592–2597, 2014, doi:10.1093/bioinformatics/btu352.
- [21] L. Zhaohui, W. Longyan, X. Jian, C. Meng, Y. Jianping, and A. C. C. Tan, "Flow reconstruction from sparse sensors based on reduced order autoencoder state estimation," *Phys. Fluids*, vol. 35, no. 7, Jul. 2023, doi: 10.1063/5.0155039.
- [22] I. Lavie, G. Gur-Ari, and Z. Ringel, "Towards understanding inductive bias in transformers: A view from infinity," *arXiv preprint arXiv:2402.05173*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.05173>
- [23] A. De and G. D. Horwitz, "Spatial receptive field structure of double-opponent cells in macaque V1," *J. Neurophysiol.*, vol. 125, no. 3, pp. 843–857, Jan. 2021, doi: 10.1152/jn.00547.2020.
- [24] T. Kattenborn, J. Leitloff, F. Schiefer, and S. Hinz, "Review on convolutional neural networks (CNN) in vegetation remote sensing," *ISPRS J. Photogramm. Remote Sens.*, vol. 173, pp. 24–49, Mar. 2021, doi: 10.1016/j.isprsjprs.2020.12.010.
- [25] G. W. Lindsay, "Convolutional neural networks as a model of the visual system: Past, present, and future," *J. Cogn. Neurosci.*, vol. 33, no. 10, pp. 2017–2031, 2021, doi: 10.1162/jocn_a_01544.
- [26] I. Karabayir, O. Akbilgic, and N. Tas, "A novel learning algorithm to optimize deep neural networks: Evolved gradient direction optimizer (EVGO)," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 2, pp. 685–694, Feb. 2021, doi: 10.1109/tnnls.2020.2979121.
- [27] S. Ojha and M. Sharma, "U-Net based image segmentation drawbacks in medical images: A review," in *Proc. Int. Conf. Image Process.*, pp. 361–372, Jan. 2024, doi: 10.1007/978-981-97-1111-6_29.
- [28] M. N. U. Laskar, L. G. Sanchez Giraldo, and O. Schwartz, "Deep neural networks capture texture sensitivity in V2," *J. Vis.*, vol. 20, no. 7, p. 21, Jul. 2020, doi: 10.1167/jov.20.7.21.
- [29] A. Arista, T. Theresiawati, and H. B. Seta, "Big Mart sales data visualization and correlation," *JOIV Int. J. Inform. Vis.*, vol. 8, no. 2, pp. 576–582, May 2024, doi: 10.62527/joiv.8.2.1780.
- [30] R. Azad et al., "Medical image segmentation review: The success of U-Net," *IEEE Trans. Pattern Anal. Mach. Intell.*, pp. 1–20, Jan. 2024, doi: 10.1109/tpami.2024.3435571.
- [31] R. Shao, Z. Zheng, H. Tu, B. Liu, H. Zhang, and Y. Liu, "Tensor4D: Efficient neural 4D decomposition for high-fidelity dynamic reconstruction and rendering," in *Proc. CVPR*, pp. 16632–16642, 2023, doi: 10.1109/CVPR52729.2023.01596.
- [32] L. Motamedpour, A. Dalimi, M. Pirestani, and F. Ghaffarifar, "In silico analysis and expression of a new chimeric antigen as a vaccine candidate against cutaneous leishmaniasis," *Iran. J. Basic Med. Sci.*, vol. 23, no. 11, pp. 1409–1418, Nov. 2020, doi:10.22038/ijbms.2020.45394.10561.
- [33] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015* (N. Navab, J. Hornegger, W. Wells, and A. Frangi, Eds.), Lecture Notes in Computer Science, vol. 9351. Cham, Switzerland: Springer, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.
- [34] A. Makhzani and B. Frey, "Sparse autoencoders," *arXiv preprint arXiv:1312.5663*, 2013.